PLANT-A-VIRTUAL-CTO — ENGINEERING GOVERNANCE MANUAL

Plant-A-Virtual-CTO Framework (SDGE)

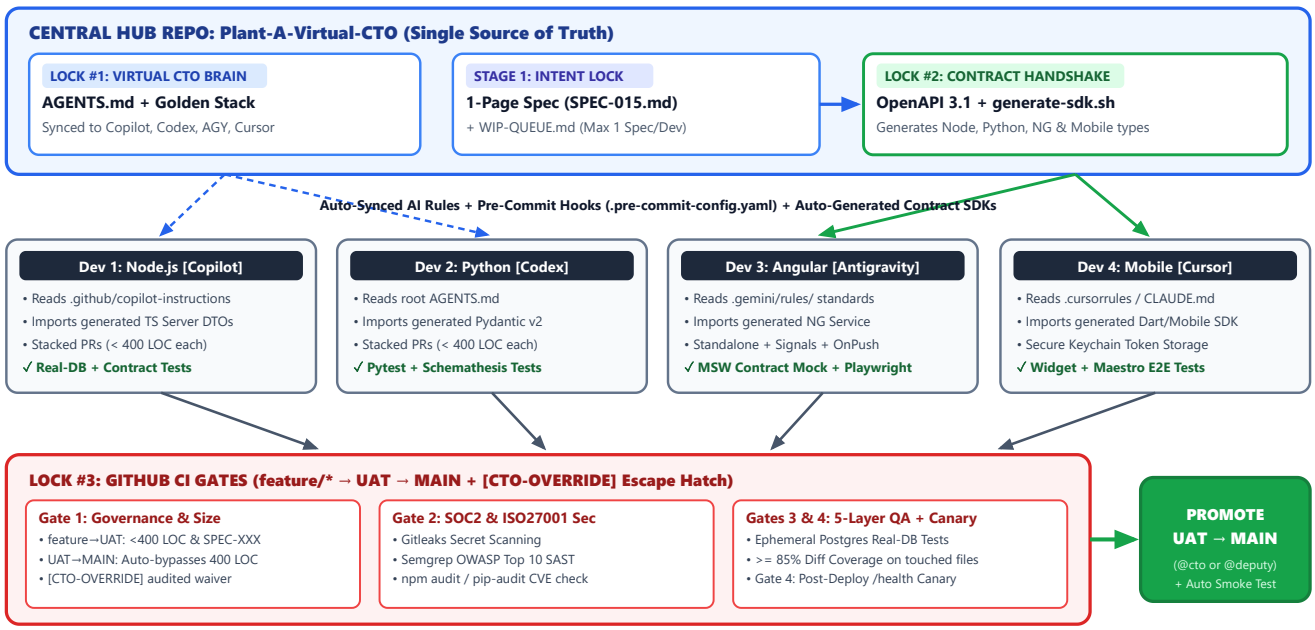
Planting a Virtual CTO Inside Every Repository — Maintaining Existing Production Services ("Service A"), Building New Services ("Service B"), UAT → MAIN Promotion & 5-Layer Testing

BRANCH & GATE FLOW	AI TOOL SUPPORT	GOLDEN STACK	COMPLIANCE & QA
feature/* → UAT → MAIN	Copilot, Codex, Antigravity, Cursor, Claude	Node/TS, Python, Angular, Mobile	SOC 2, ISO 27001 & 5-Layer Testing

1. Executive Overview & Multi-Developer Synchronization Architecture

When developers work independently with their own choice of AI coding tools (GitHub Copilot, OpenAI Codex, Antigravity, Cursor, Claude Code), **code generation is no longer the bottleneck—alignment, contract consistency, and verification are.** Instead of Agile ceremonies, **Plant-A-Virtual-CTO** uses **3 Automated Synchronization Locks** to keep every developer and every AI tool aligned across all repositories:

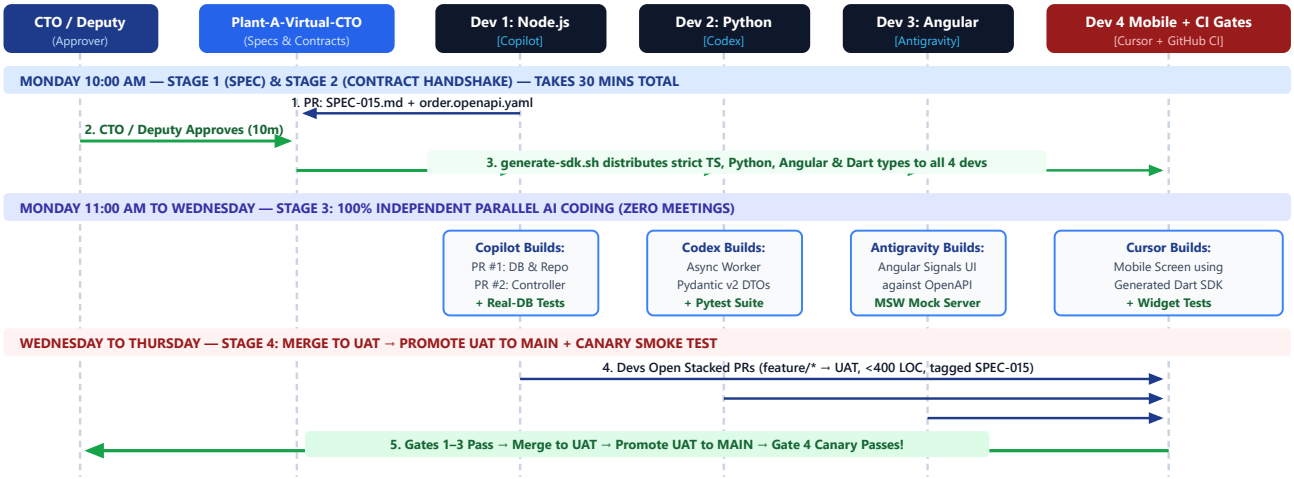
DIAGRAM 1: THE 3 SYNCHRONIZATION LOCKS — HOW "PLANT-A-VIRTUAL-CTO" ALIGNS 4 DEVELOPERS & 4 AI TOOLS



1.1 Multi-Repository Hub Topology & Monday-to-Friday Synchronization Sequence

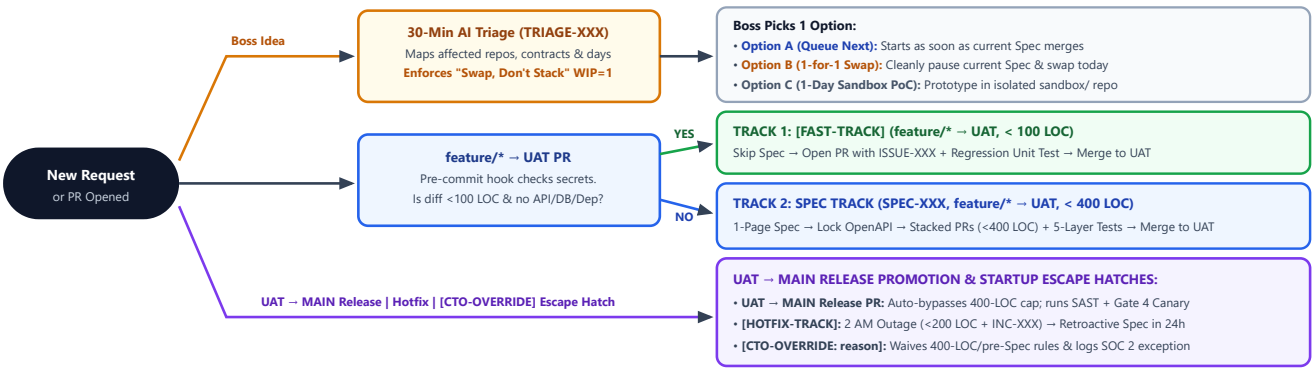
Repository	Role	What Lives Inside
`Plant-A-Virtual-CTO`	**Central Governance & Contract Hub**	`AGENTS.md`, `specs/WIP-QUEUE.md`, `specs/SPEC-XXX.md`, `contracts/openapi/`, `contracts/events/`, `docs/adr/`, and sync scripts (`sync-microservice.ps1` / `.sh`).
`service-a` (Brownfield)	**Existing Production Microservice**	Synced copies of `AGENTS.md`, AI tool configs, `.pre-commit-config.yaml`, `CODEOWNERS` (`@cto @deputy`), PR Template, and Diff-Only CI Gates.
`service-b` (Greenfield)	**New Microservice Built From Scratch**	Bootstrapped with `sync-microservice` on Day 1; 100% Golden Stack compliance, consuming generated OpenAPI/Event models from `Plant-A-Virtual-CTO`.
`web-angular` & `mobile-app`	**Client Applications**	Consume auto-generated TypeScript (Angular) and Dart/React-Native (Mobile) client SDKs generated from `contracts/openapi/`.

DIAGRAM 2: MONDAY-TO-FRIDAY SEQUENCE — SYNCHRONIZING 4 DEVELOPERS ACROSS BACKEND, AI, ANGULAR & MOBILE



2. Branch Promotion (`feature/* → UAT → MAIN`), Tracks & Escape Hatch

DIAGRAM 3: WORK INTAKE, UAT → MAIN PROMOTION & STARTUP "[CTO-OVERRIDE]" ESCAPE HATCH



3. Scenario 1: Maintaining "Service A" (Existing Production Service)

Context: `Service A` is already live in production. It contains legacy code written with different AI tools, existing database tables, and undocumented REST endpoints.

> **The Golden Rule for Brownfield Services:** Do not freeze feature work to rewrite `Service A` or write OpenAPI contracts for all 50+ existing endpoints at once. Onboard `Service A` in 5 minutes and apply the **"Touch It, Standardize It"** rule.

Step 0: One-Time Onboarding of `Service A` (Takes 5 Minutes)

Clone `Plant-A-Virtual-CTO` and `service-a` side-by-side and run the sync script:

POWERSHELL / BASH COMMAND

```
# Windows PowerShell (set -Stack to "node", "python", "angular", or "mobile")
.\scripts\sync-microservice.ps1 -TargetRepoPath "..\service-a" -Stack "node"

# macOS / Linux / Git Bash
./scripts/sync-microservice.sh ../service-a node
```

What this does to `Service A` without breaking existing code:

- Plants the Virtual CTO (`AGENTS.md` , `.github/copilot-instructions.md` , `.gemini/rules/Plant-A-Virtual-CTO.md` , `.cursorrules` , and `CLAUDE.md`) so every developer's AI tool immediately stops copying legacy anti-patterns.
- Installs `.pre-commit-config.yaml` (local 3-second Gitleaks secret check + linter before `git push`) and `.github/CODEOWNERS` (`@planta94 @deputy-lead-handle`).
- Installs `.github/workflows/ci-gates.yml` supporting `feature/* → UAT → MAIN` , `[CTO-OVERRIDE]` , Diff-Only Linting/SAST/Coverage (`≥ 85%`), and Gate 4 Post-Deploy Canary checks.

Case 1A: Fixing a Small Bug or Tweak in `Service A` (< 100 LOC)

COPY-PASTE AI PROMPT (FAST-TRACK BUGFIX)

```
Read AGENTS.md. Fix the bug in src/services/billing.service.ts where [describe bug].
Strict constraints:
1. Do NOT reformat or refactor unrelated functions in this legacy file.
2. Do NOT add any new packages to package.json / pyproject.toml.
3. Follow the Golden Stack rules in AGENTS.md for the lines you touch and add a regression unit test.
4. Keep the total git diff under 100 lines.
```

Case 1B: Adding a Feature or Modifying an Existing Endpoint in `Service A`

Example: Adding discount code support to an existing production endpoint `POST /v1/orders` in `Service A` .

Stage 1 — Draft & Approve a 1-Page Spec (10 Minutes)

COPY-PASTE AI PROMPT (STAGE 1 SPEC)

```
Read AGENTS.md and specs/TEMPLATE-1PAGE-SPEC.md. Draft specs/SPEC-012-order-discounts.md to add optional
discountCode support to POST /v1/orders in service-a. Include backward-compatibility rules for live
Angular/Mobile clients, Expand-and-Contract DB migration steps, SOC 2 tenant isolation checks, Section 6
Mandatory Test Matrix, and a breakdown of stacked PRs under 400 LOC each.
```

Stage 2 — "Touch It, Contract It" (Extract Baseline & Lock Contract First)

COPY-PASTE AI PROMPT (STAGE 2 BROWNFIELD CONTRACT EXTRACTION)

```
Follow ai-context/reverse-engineer-contract.skill.md. Read src/controllers/orders.controller.ts in
service-a, extract the current production request/response schema for POST /v1/orders into
contracts/openapi/service-a.openapi.yaml, and add the new optional discountCode field in a backward-
compatible way.
```

Stage 3 — Implement in Stacked PRs (`feature/* → UAT` , < 400 LOC Each) & Promote `UAT → MAIN`

PR #1: DB Migration & Real-DB Repo Tests (~220 LOC)

Adds a **nullable** column (Expand-and-Contract pattern so live production instances don't break during rollout) + repository query + real-DB integration test. Merges to `UAT` .

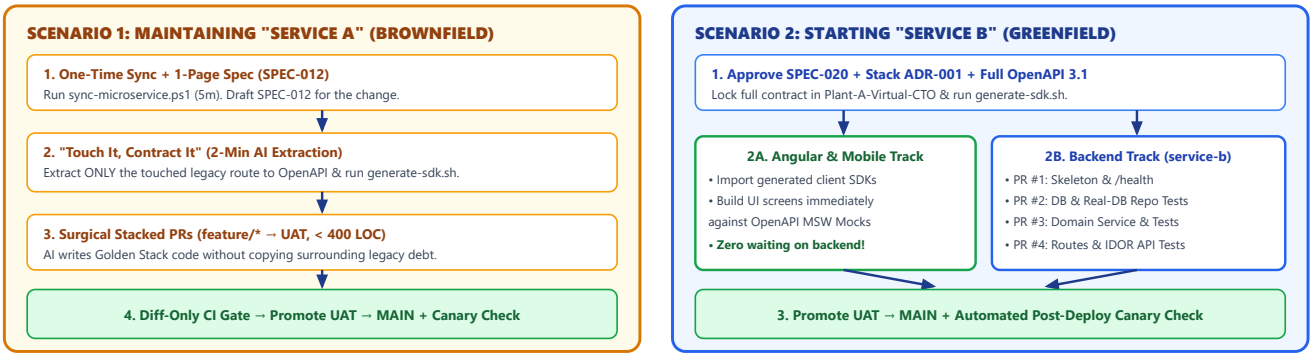
PR #2: Service, Controller & Contract/IDOR Tests (~260 LOC)

Implements discount validation, wires the controller using generated OpenAPI DTOs, and adds HTTP contract + 403 IDOR tests. Merges to `UAT` , then promotes `UAT → MAIN` .

4. Scenario 2: Starting "Service B" From Scratch (Greenfield Service)

Context: You are launching a brand-new microservice (`Service B`). Starting from scratch with **Plant-A-Virtual-CTO** allows your backend, Angular, and Mobile developers to work **100% in parallel** with zero integration mismatches.

DIAGRAM 4: SIDE-BY-SIDE WORKFLOW — MAINTAINING "SERVICE A" (BROWNFIELD) VS. STARTING "SERVICE B" (GREENFIELD)



Step-by-Step Prompts to Build `Service B` in 4 Stacked PRs (< 400 LOC Each)

After running `.\scripts\sync-microservice.ps1 -TargetRepoPath "..\service-b" -Stack "python"` , build `Service B` using these 4 sequential prompts:

PR #1 PROMPT — SKELETON, CONFIG & RFC 7807 ERROR HANDLER (~200 LOC)

```
Read AGENTS.md and specs/SPEC-020-service-b.md. Implement ONLY PR #1 (Service Skeleton):
1. Set up folder structure (controllers/, services/, repositories/, config/, tests/).
2. Configure strict startup environment validation, structured logging (redacting auth/PII), global RFC
7807 ProblemDetails exception handler, GET /health, and healthcheck test. Reference SPEC-020 and ADR-001.
Keep diff under 250 LOC.
```

PR #2 PROMPT — DATABASE SCHEMA, MIGRATIONS & REAL-DB REPO TESTS (~250 LOC)

```
Read AGENTS.md and specs/SPEC-020-service-b.md. Implement ONLY PR #2 (Persistence Layer):
1. Create the initial DB migration and typed ORM models (Prisma / SQLAlchemy 2.0 Async).
2. Implement repositories/ with parameterized queries and real-DB integration tests (do not mock the ORM).
Keep diff under 300 LOC.
```

PR #3 PROMPT — DOMAIN BUSINESS LOGIC & EDGE-CASE UNIT TESTS (~250 LOC)

```
Read AGENTS.md and specs/SPEC-020-service-b.md. Implement ONLY PR #3 (Domain Service Layer):
1. Implement business logic in services/ using the repository interfaces from PR #2.
2. Add unit tests covering all failure modes and edge cases in Section 6 of SPEC-020 (≥ 85% diff
coverage). Keep diff under 300 LOC.
```

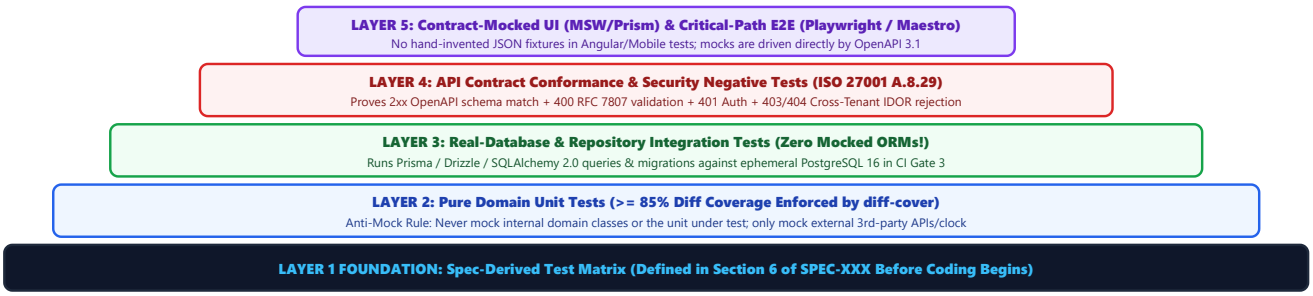
PR #4 PROMPT — API CONTROLLERS, AUTH GUARDS & CONTRACT/IDOR TESTS (~250 LOC)

```
Read AGENTS.md and contracts/openapi/service-b.openapi.yaml. Implement ONLY PR #4 (Transport &
Verification):
1. Wire HTTP routes importing generated contract DTOs. Enforce authentication and tenant-isolation guards.
2. Write contract integration tests verifying 200/201 schema conformance, 400 RFC 7807 validation, 401
unauthenticated, and 403/404 cross-tenant IDOR rejection. Keep diff under 300 LOC.
```

5. Full-Fledged 5-Layer Testing Architecture (Anti-AI-Hallucination)

When developers ask AI tools to "*Write unit tests*" after writing code, AI defaults to the **"AI Mock Trap"**—mocking the ORM/functions it just wrote and asserting the mock returns itself. Our framework enforces a **5-Layer Testing Trophy** in `AGENTS.md` and CI `Gate 1` & `Gate 3`:

DIAGRAM 5: THE 5-LAYER TESTING TROPHY & AUTOMATED CI VERIFICATION PIPELINE



6. Golden Stack & Developer Contribution Loop (`AGENTS.md`)

Stack	Mandatory Golden Stack Standards	Explicitly Banned Patterns (Blocked in Review/CI)
Node.js / TypeScript	Node LTS, TypeScript (`strict: true` , `noImplicitAny: true`), NestJS or Fastify, Prisma/Drizzle ORM, Zod, Vitest/Jest + Supertest + Testcontainers.	❌ `any` or `@ts-ignore` , `axios` , `moment.js` ❌ Direct ORM calls in Controllers or mocked ORMs in repo tests
Python Services	Python 3.11+, FastAPI, Pydantic v2 (`model_validate` , `extra="forbid"`), SQLAlchemy 2.0 Async + Alembic, `ruff` , `mypy --strict` , `pytest` + `schemathesis` .	❌ Untyped function signatures, Pydantic v1 `.dict()` ❌ Sync `requests.get` inside `async def` , bare `except: pass`
Angular Frontend	Standalone Components (`standalone: true`), Angular Signals (`signal` , `computed`), `ChangeDetectionStrategy.OnPush` , OpenAPI-generated services, MSW + Playwright.	❌ Legacy `NgModule` , hand-written duplicate DTOs or fake JSON mocks ❌ Function calls in HTML templates, unsubscribed observables
Mobile Apps	OpenAPI-generated Dart/TS client models, clean `Presentation → Domain → Data` separation, hardware-backed secure storage, Maestro E2E.	❌ Direct HTTP calls or business logic inside UI Widgets/Screens ❌ Storing tokens in plain `SharedPreferences` / `AsyncStorage`

6.1 How Developers Teach the Virtual CTO ("Compound AI Memory Loop")

Whenever a developer catches their AI tool making a mistake or discovers a better pattern/prompt, they contribute it back to `Plant-A-Virtual-CTO` so every developer's AI tool learns immediately:

DIAGRAM 6: THE DEVELOPER-DRIVEN "COMPOUND AI MEMORY" CONTRIBUTION LOOP



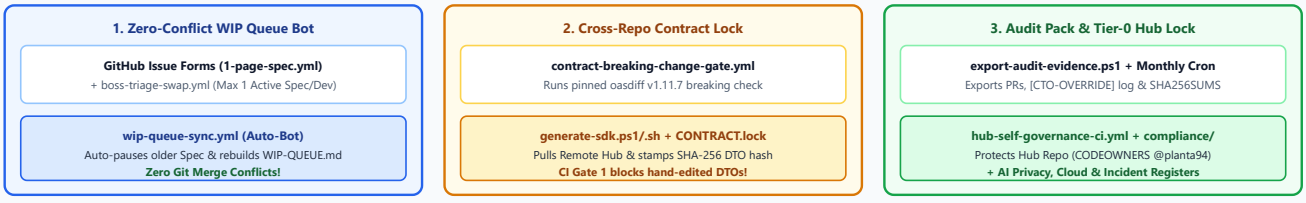
7. SOC 2 & ISO/IEC 27001:2022 Automated Control Matrix

Compliance Control	Control Objective	Automated Enforcement & Audit Artifact in Plant-A-Virtual-CTO
SOC 2 CC8.1 **ISO 27001 A.8.32**	Change Management, Authorization & Exception Logging	**CI Gate 1 (`change-control-gate`)** enforces `SPEC-XXX` or `[FAST-TRACK]` on `feature/* → UAT`, auto-detects `UAT → MAIN` releases, and logs any `[CTO-OVERRIDE: reason]` exception. `CODEOWNERS` (`@cto @deputy`) enforces Segregation of Duties.
ISO 27001 A.8.25 & A.8.28	Secure SDLC, Laptop Pre-Commit & Secure Coding	Codified in `AGENTS.md` and synced to Copilot, Antigravity, Codex, Cursor, and Claude. Local `.pre-commit-config.yaml` blocks secrets in 3s on developer laptops before `git push`.
ISO 27001 A.8.26 & A.8.31	Application Security & Environment Separation (UAT vs MAIN)	Sections 5 & 6 of `specs/TEMPLATE-1PAGE-SPEC.md` mandate sign-off on Auth, Tenant Isolation (IDOR), PII, and Test Matrix. Strict separation between `UAT` and `MAIN` branches.
ISO 27001 A.8.29 **SOC 2 CC7.1 & CC7.2**	Security Testing, 5-Layer QA & Post-Deploy Canary Monitoring	**CI Gate 1** blocks code PRs missing tests. **CI Gate 2** runs **Gitleaks**, **Semgrep SAST**, and **CVE audits**. **CI Gate 3** runs real-DB tests (`≥ 85%` diff coverage). **CI Gate 4** runs post-deploy Canary `/health` smoke probes on `UAT` and `MAIN`.
ISO 27001 A.5.12 **SOC 2 Confidentiality**	AI Data Privacy & Protection of PII/Secrets	CI Gate 1 enforces mandatory PR checkbox attestation: "`[x]` *No production secrets, customer PII, or proprietary credentials were shared with external AI tools or committed.*"

8. Enterprise Hardening: Zero-Conflict WIP Sync, Contract Safety, Audit Pack & Hub Self-Governance

To ensure `Plant-A-Virtual-CTO` rivals commercial governance platforms without recurring SaaS costs, the repository includes four automated hardening subsystems:

DIAGRAM 7: ENTERPRISE HARDENING ARCHITECTURE (WIP BOT, CONTRACT LOCK, AUDIT EXPORTER & HUB TIER-0 LOCK)



Hardening Subsystem	Files & Automation Workflows	What It Enforces Automatically
1. Zero-Conflict WIP Queue & Automatic Swap	`.github/ISSUE_TEMPLATE/1-page-spec.yml` `.github/ISSUE_TEMPLATE/boss-triage-swap.yml` `.github/workflows/wip-queue-sync.yml`	Developers create/swap Specs via GitHub Issue Forms. The bot automatically enforces **Max 1 Active Spec/Dev** (auto-pausing older Specs on swap), auto-updates status when PRs merge to `UAT` / `MAIN`, and commits `specs/WIP-QUEUE.md` on `main` with zero merge conflicts.
2. Cross-Repo Contract Safety & SHA-256 Lock	`.github/workflows/contract-breaking-change-gate.yml` `scripts/generate-sdk.ps1` & `.sh` `CONTRACT.lock`	Runs pinned **`oasdiff v1.11.7 breaking`** on OpenAPI PRs and blocks backward-incompatible field deletions/renames. Supports remote Hub fetching (`-RemoteHubUrl`) and stamps a SHA-256 `CONTRACT.lock` verified by Microservice CI Gate 1.
3. Full SOC 2 & ISO 27001 Evidence & Registers	`scripts/export-audit-evidence.ps1` & `.sh` `.github/workflows/monthly-audit-evidence.yml` `compliance/*.md`	Exports tamper-evident `SHA256SUMS.txt` audit bundles (production PRs, `[CTO-OVERRIDE]` register, branch protection, collaborator roster) + covers the remaining 80% of the audit (AI Zero-Training Register, Quarterly Access/Cloud/Laptop Checklist, and Incident Response Playbook).
4. Hub Self-Governance (Tier-0 Protection)	`.github/CODEOWNERS` `.github/workflows/hub-self-governance-ci.yml` `SECURITY.md` & `.pre-commit-config.yaml`	Protects `Plant-A-Virtual-CTO` itself: locks workflows, `AGENTS.md`, scripts, and contracts to `@planta94` (and Deputy Lead); runs Gitleaks, PR attestation, Bash/YAML/JSON syntax & UTF-8 checks on every Hub PR.